

Fiche de synthèse globale — Cloud & Azure (TP1 → TP4)

Synthèse dédoublonnée des 4 cours magistraux du Bloc 4. Pensée pour réviser un **devoir sur table** (mise en situation, choix d'architecture et techniques). Fil conducteur : **ShopEasy**.

0. Le fil conducteur : un cycle de vie en 4 temps

TP	Posture	Question centrale	Outils clés
TP1 — Concevoir	Architecte	Quelle architecture pour le besoin ?	Services Azure, Well-Architected
TP2 — Industrialiser	DevOps / IaC	Comment la rendre reproductible ?	Terraform (HCL, state, plan)
TP3 — Administrer	Ops / Exploitant	Comment la piloter au quotidien ?	Azure CLI, Bash, Python SDK
TP4 — Piloter	SRE / FinOps / RSSI	Est-elle fiable, sûre et rentable ?	Monitor, Cost Mgmt, Defender

Idée maîtresse : le cloud ne garantit pas une bonne architecture. Une plateforme mature repose sur 4 piliers opérationnels : **observabilité, FinOps, sécurité, gouvernance**. Provisionner ≠ exploiter.

1. Fondamentaux du cloud

Cloud = ressources informatiques **à la demande**, accessibles par le réseau, mesurées, facturées à l'usage. **4 propriétés** : élasticité · facturation à l'usage (CAPEX→OPEX, ⚠ ressource oubliée = coût) · automatisation · services managés.

IaaS / PaaS / SaaS — arbitrage **contrôle** ↔ **responsabilité** ↔ **charge d'admin** :

Modèle	Le client gère	Exemple Azure
IaaS	OS, runtime, app, données, sécu	VM, Virtual Network
PaaS	code, config, données, droits	App Service, Azure SQL Database
SaaS	usage, données	Microsoft 365

Modèle de responsabilité partagée : Azure sécurise l'infra physique + plateforme ; le **client** reste responsable de la config, des identités, des données, des accès et de l'architecture. *Utiliser Azure ne sécurise pas automatiquement.*

Région vs Availability Zone : région = zone géographique (latence, conformité, résidence des données) ; AZ = datacenters **physiquement séparés** dans une région (résilience face à la panne d'un datacenter).

2. Organisation & gouvernance Azure

Hiérarchie : Management Group → **Tenant** (identité, Entra ID) / **Subscription** (facturation + droits) → **Resource Group** (cycle de vie d'une app/env) → **Resource** → **Tags**.

Leviers de gouvernance : nommage cohérent · **Tags** · **Azure Policy** (imposer/auditer des règles, ex. refuser une ressource sans tag) · **RBAC**.

Tags recommandés : **Environment**, **Owner**, **Application**, **CostCenter**, **Criticality**, **DataSensitivity**.

RBAC & moindre privilège : rôles (**Owner** admin total → à limiter, **Contributor** gère sans les droits, **Reader** lecture, **Cost Management Reader**) appliqués sur un **scope** (MG → sub → RG → resource). ⚠
Jamais Owner par facilité.

Antiséche gouvernance : un environnement **sans tags ni convention** est ingérable (coûts non attribuables, audit difficile, ressources orphelines). Les tags sont simples mais centraux pour FinOps + audit + automatisation.

3. Briques d'architecture applicative

Réseau

- **VNet** = réseau privé isolé (ex. **10.10.0.0/16**).
- **Subnets** = segmentation par fonction/exposition (web / data / admin) → limite les **mouvements latéraux**.
- **NSG** = filtrage entrant/sortant (priorité, source, port, action).

Règles type (à mémoriser) :

Port	Flux	Source autorisée
80 / 443	HTTP / HTTPS	Internet (via LB)
22	SSH	IP admin uniquement (ou Bastion)
3389	RDP	IP admin (jamais 0.0.0.0/0)
1433	SQL	subnet applicatif uniquement

Compute

- **VM (IaaS)** : contrôle total, idéale pour migration « lift-and-shift ». Bien **dimensionner**.
- **App Service (PaaS)** : moins d'admin OS/patches, pour app web standard.
- **Disponibilité** : plusieurs instances derrière un **Load Balancer** + sondes de santé (+ multi-AZ).

Équilibrage

	Load Balancer	Application Gateway
Couche	4 (TCP/UDP)	7 (HTTP/HTTPS)

	Load Balancer	Application Gateway
Plus	simple, réseau	TLS, routage URL, WAF

⚠ Un LB **seul** ne rend pas HA : HA = instances + zones + sondes + sauvegardes + supervision.

Stockage & Données

- **Storage Account / Blob** : objets (documents, images, archives). Bonnes pratiques : **privé**, versioning, cycle de vie, TLS 1.2.
- **Managed Disks** (disques VM), **Azure Files** (partage SMB/NFS).
- **Azure SQL Database (PaaS)** vs **SQL sur VM (IaaS)** : managé (sauvegardes/HA/patches gérés) vs contrôle total. La base **ne s'expose jamais** directement à Internet (subnet data / Private Endpoint).

4. Le cadre d'évaluation : Well-Architected Framework

Les **5 piliers** = la grille pour **justifier et critiquer** une architecture :

Pilier	Question
Reliability	Et si une ressource tombe ? Sauvegarde/reprise ? SPOF ?
Security	Accès limités ? Données protégées ? Flux filtrés ? Audit ?
Cost Optimization	Bon dimensionnement ? Coûts suivis ? Env inutiles arrêtés ?
Operational Excellence	Exploitation standardisée ? Changements tracés ? Supervision ?
Performance Efficiency	Ressources adaptées ? Absorbe la croissance ?

Lecture critique (8 réflexes) : ressources exposées identifiées ? base protégée ? supervision ? coûts estimés/attribuables ? **SPOF** ? droits limités ? données sauvegardées/versionnées ? choix justifiés par le besoin métier ?

5. Industrialiser : Infrastructure as Code (Terraform)

IaC = décrire l'infra en fichiers versionnés (source de vérité) → reproductibilité, traçabilité (Git), standardisation, destruction propre. **Terraform** = outil **déclaratif** (on décrit l'état voulu) et **multi-cloud** (providers, ex. **azurerm**).

Workflow : **init** → **fmt** → **validate** → **plan** → **apply** → **output** → **destroy**. **Symboles du plan** : **+** créer · **-** détruire · **~** modifier · **-/+** **remplacer** (destruction+recréation ⚠).

Notions clés : - **Variable** (entre) / **Local** (calculé) / **Output** (exposé après déploiement). - **State** (**tfstate**) = lien code ↔ ressources réelles. ⚠ **jamais dans Git**, peut contenir des secrets → **backend distant** (Storage Account + verrouillage) en équipe. - **Drift** = écart entre code et réel (souvent une modif manuelle au portail) ; **plan** le détecte → tout passe par le code, pas par le portail. - **Modules** = blocs réutilisables (réseau, VM...). - **Secrets** : jamais dans **.tf** / **.tfvars** versionnés → Key Vault, variables sécurisées CI/CD.

6. Administrer & automatiser (CLI / Bash / Python)

Azure CLI : préféré au portail car **rejouable, documentable, scriptable**. Sorties `-o json|table|tsv|yaml|none`. `--query (JMESPath)` filtre/transforme le JSON.

Bash : script fiable = `#!/usr/bin/env bash` + `set -euo pipefail`, variables centralisées, **idempotent**, sorties conservées.

Python (SDK) : quand le besoin grandit (traitement structuré, API, rapports). Libs : `azure-identity` (`DefaultAzureCredential`), `azure-mgmt-resource/compute/monitor`.

⚠ **az vm stop vs az vm deallocate (piège classique)** : `stop` arrête mais **facture encore le compute** ; `deallocate` **libère la capacité de calcul** → **coupe le coût compute** (les disques restent facturés).

Inventaire = base de l'exploitation : savoir ce qui existe avant de surveiller/sécuriser/optimiser.

7. Piloter : monitoring & observabilité

4 questions complémentaires : Monitoring (*ça marche ?*) · Observabilité (*pourquoi ?*) · Audit (*qui a fait quoi ?*) · Sécurité (*est-ce protégé ?*). **3 signaux : Métriques** (numériques → alerte sur seuil) · **Logs** (événements → diagnostic) · **Traces** (chemin d'une requête).

Azure Monitor : Metrics, Logs, **Log Analytics Workspace** (KQL), Alert Rules, **Action Groups**, Workbooks, Dashboards, **Activity Log** (opérations de gestion).

Stratégie : ne pas tout surveiller. *Avant de créer une alerte : si elle se déclenche, quelqu'un doit-il agir ?* Si non → dashboard, pas alerte. **SLI < SLO < SLA** : indicateur **mesuré** < objectif **interne** < engagement **contractuel**. **Alerte = action** : condition + sévérité + action group + **runbook**. ⚠ **Fatigue d'alerte** : trop d'alertes = ignorées. **Cycle incident** : Détecter → Qualifier → Diagnostiquer → Corriger → Capitaliser.

8. FinOps (vue consolidée)

FinOps = finance + tech + ops → **maximiser la valeur/coût** (pas juste réduire). 3 objectifs : rendre les coûts **visibles**, **responsabiliser** (coût ↔ propriétaire), **optimiser en continu**.

Outils : Pricing Calculator (avant), Cost Management + Budgets + Cost alerts (après), Tags (ventilation), Reservations/Savings Plans (usage stable), Advisor (recommandations).

Axes d'optimisation : supprimer l'inutilisé (disques orphelins, IP), **éteindre/deallocate les VM hors-prod**, redimensionner (rightsizing), bon niveau de service, budgets + tags obligatoires.

Une bonne reco FinOps précise : *la ressource, le problème, l'impact estimé, le risque, l'action* — pas « réduire les coûts ». **Réduire** = dépenser moins. **Optimiser** = meilleur rapport coût/valeur.

9. Sécurité & audit (vue consolidée)

Piliers : responsabilité partagée · **moindre privilège** (RBAC) · sécurité réseau (NSG, fermer les ports inutiles, isoler les bases) · gestion des secrets (Key Vault).

Risques typiques → **mesures** :

Risque	Mesure corrective
SSH/RDP ouvert à Internet	Restriction IP /32, Azure Bastion , MFA
Base exposée	Accès privé, règles réseau strictes, Private Endpoint
Stockage public	Conteneur privé, SAS/identité managée, chiffrement
Droits excessifs (Owner)	RBAC minimal + revue d'accès
Absence de logs	Diagnostic settings → Log Analytics
Compte partagé	Comptes nominatifs + MFA

Defender for Cloud : pilotage de posture (**Secure Score**, recommandations, compliance). Toutes les recos ne se valent pas → **prioriser** (exposition, sensibilité, coût, criticité).

Audit & preuve : sources = **Activity Log**, Resource Logs, Entra ID logs, Diagnostic settings, Azure Policy. En pro, il faut **prouver** (logs, captures, Policy), pas seulement affirmer.

10. Méthode : choisir, justifier, recommander

Partir du besoin, pas du service. Pour chaque besoin : quelle fonction ? quel niveau de service ? quelle responsabilité garder ? quel risque réduire ? quel coût acceptable ?

Matrice de décision :

Besoin	Option simple	Option cible	Critère
App web	VM	App Service / VM + LB	contrôle, migration
Base SQL	SQL sur VM	Azure SQL Database	admin, sauvegarde, dispo
Documents	Disque VM	Storage Account	durabilité, versioning
Accès admin	SSH public	Bastion / restriction IP	sécurité, audit
Monitoring	aucun	Azure Monitor	exploitabilité
Coûts	estimation manuelle	Pricing Calc + Budget	gouvernance FinOps

Note de recommandations DSI : Contexte → Constats → Risques → **Recommandations priorisées (justifiées)** → Plan d'action (court/moyen/long) → Indicateurs de suivi. La DSI veut une **décision lisible et priorisée** reliant technique ↔ risque ↔ coût ↔ valeur métier.

11. Tableaux « antisèche » (à revoir juste avant l'épreuve)

Ports & sources : 80/443 ← Internet · 22/3389 ← IP admin · 1433 ← subnet web.

Couche réseau : Load Balancer = **L4** · Application Gateway = **L7 (WAF/TLS)**.

IaaS/PaaS : VM/VNet/NSG = IaaS · App Service/Azure SQL/Storage = PaaS · M365 = SaaS.

Coût VM : **stop** = facture compute · **deallocate** = **coupe le compute**.

Terraform : **-/+** = remplacement (danger) · state hors Git · drift = modif manuelle.

Niveaux d'engagement : SLI (mesuré) < SLO (interne) < SLA (contrat).

Erreurs classiques à ne JAMAIS commettre : SSH `0.0.0.0/0` · base exposée · stockage public · Owner pour tous · compte partagé · pas de tags · pas de supervision · SPOF (1 seule VM) · modif manuelle au portail (drift) · secrets dans Git.

Note environnement (Azure for Students) : `francecentral` refusée → `swedencentral` ;
`Standard_B1s` indisponible → `Standard_B2ts_v2`.